

Contents		1202 LeetCode		1203 LeetCode		1204 LeetCode		1205 LeetCode		1206 LeetCode		1207 LeetCode		1208 LeetCode		1209 LeetCode		1210 LeetCode		1211 LeetCode		1212 LeetCode		1213 LeetCode		1214 LeetCode		1215 LeetCode		1216 LeetCode		1217 LeetCode		1218 LeetCode		1219 LeetCode		1220 LeetCode		1221 LeetCode		1222 LeetCode		1223 LeetCode		1224 LeetCode		1225 LeetCode		1226 LeetCode		1227 LeetCode		1228 LeetCode		1229 LeetCode		1230 LeetCode		1231 LeetCode		1232 LeetCode		1233 LeetCode		1234 LeetCode		1235 LeetCode		1236 LeetCode		1237 LeetCode		1238 LeetCode		1239 LeetCode		1240 LeetCode		1241 LeetCode		1242 LeetCode		1243 LeetCode		1244 LeetCode		1245 LeetCode		1246 LeetCode		1247 LeetCode		1248 LeetCode		1249 LeetCode		1250 LeetCode		1251 LeetCode		1252 LeetCode		1253 LeetCode		1254 LeetCode		1255 LeetCode		1256 LeetCode		1257 LeetCode		1258 LeetCode		1259 LeetCode		1260 LeetCode		1261 LeetCode		1262 LeetCode		1263 LeetCode		1264 LeetCode		1265 LeetCode		1266 LeetCode		1267 LeetCode		1268 LeetCode		1269 LeetCode		1270 LeetCode		1271 LeetCode		1272 LeetCode		1273 LeetCode		1274 LeetCode		1275 LeetCode		1276 LeetCode		1277 LeetCode		1278 LeetCode		1279 LeetCode		1280 LeetCode		1281 LeetCode		1282 LeetCode		1283 LeetCode		1284 LeetCode		1285 LeetCode		1286 LeetCode		1287 LeetCode		1288 LeetCode		1289 LeetCode		1290 LeetCode		1291 LeetCode		1292 LeetCode		1293 LeetCode		1294 LeetCode		1295 LeetCode		1296 LeetCode		1297 LeetCode		1298 LeetCode		1299 LeetCode		1300 LeetCode		1301 LeetCode		1302 LeetCode		1303 LeetCode		1304 LeetCode		1305 LeetCode		1306 LeetCode		1307 LeetCode		1308 LeetCode		1309 LeetCode		1310 LeetCode		1311 LeetCode		1312 LeetCode		1313 LeetCode		1314 LeetCode		1315 LeetCode		1316 LeetCode		1317 LeetCode		1318 LeetCode		1319 LeetCode		1320 LeetCode		1321 LeetCode		1322 LeetCode		1323 LeetCode		1324 LeetCode		1325 LeetCode		1326 LeetCode		1327 LeetCode		1328 LeetCode		1329 LeetCode		1330 LeetCode		1331 LeetCode		1332 LeetCode		1333 LeetCode		1334 LeetCode		1335 LeetCode		1336 LeetCode		1337 LeetCode		1338 LeetCode		1339 LeetCode		1340 LeetCode	
----------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--	---------------	--

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Two Pointers #116 Sum Closest LeetCode - Simplified - WACC - LeetCode Array, Two Pointers - Sorting Line-Code 100 Time Complexity: O(n^2) Space Complexity: O(1) or O(n) depending on implementation Problem Given an integer array <code>nums</code> of length <code>n</code> and an integer <code>target</code>, find three integers in <code>nums</code> such that the sum is closest to <code>target</code>. Return the sum of the three integers. Do not assume that each input would have exactly one solution. Example 1 Input: <code>nums = [-1, 2, 1, -4]</code>, <code>target = 1</code> Output: <code>-2</code> Explanation: The sum of <code>-1</code>, <code>2</code>, and <code>-4</code> is <code>-3</code>, which is the closest to <code>target</code>. Example 2 Input: <code>nums = [0, 0, 0]</code>, <code>target = 0</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(0, 0, 0)</code>. Example 3 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 4 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 5 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 6 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 7 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 8 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 9 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 10 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 11 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 12 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 13 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 14 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 15 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 16 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 17 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 18 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 19 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 20 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 21 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 22 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 23 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 24 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 25 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 26 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 27 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 28 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 29 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 30 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 31 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 32 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 33 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 34 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 35 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 36 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 37 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 38 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 39 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 40 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code> Explanation: The only triplet that sums up to <code>0</code> is <code>(1, 1, -1)</code>. Example 41 Input: <code>nums = [1, 1, 1, 0]</code>, <code>target = 1</code> Output: <code>0</code></
--

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

Time & BST #112 Path Sum LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Given the root of a binary tree, return the root of the tree if it is a valid binary tree. A valid binary tree is one where the root is the only node with no parent. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #226 Invert Binary Tree LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Given the root of a binary tree, invert the tree, and return its root. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #230 Kth Smallest Element in a BST LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Given the root of a binary tree, return the kth smallest element in the tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #235 Binary Search Tree Iterator LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Implement a BST iterator that returns the next smallest element in the tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #236 Lowest Common Ancestor of a Binary Tree LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Given two nodes in a binary tree, return their lowest common ancestor. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #237 Delete Node in a Binary Search Tree LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Delete a node from a binary search tree and return the root of the tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #238 Binary Search Tree Validator LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Validate if a binary tree is a valid binary search tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #239 Sliding Window Maximum LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Find the maximum value in each sliding window of size k. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #240 Search a 2D Matrix LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(1) Problem Search for a target value in a 2D matrix. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #241 Different Ways to Add Parentheses LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Given an expression with numbers and operators, return all possible ways to evaluate it. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #242 Binary Search Tree Validator LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Validate if a binary tree is a valid binary search tree. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #243 Shortest Distance from All Buildings LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Find the shortest distance from a house to all buildings. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7))</pre> Example 2: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> 4((4)) 2 --> 5((5)) 3 --> 6((6)) 3 --> 7((7)) 4 --> 8((8)) 4 --> 9((9))</pre>	Time & BST #244 Shortest Distance from All Buildings LeetCode - Simple - WACC - LeetCode Time: O(N) Space Complexity: O(N) Problem Find the shortest distance from a house to all buildings. Example 1: <pre>graph TD 1((1)) --> 2((2)) 1 --> 3((3)) 2 --> </pre>
---	--	--	---	---	---	--	--	---	---	--	---	--

1045. Current Salary (E, M, H) Given a list of salaries, find the current salary of an employee on the basis of the list. Input: salaries = [1000, 2000, 3000, 4000, 5000] Output: 4000 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: salaries = [1000, 2000, 3000, 4000, 5000] Output: 4000</pre> Example 2: <pre>Input: salaries = [1000, 2000, 3000, 4000, 5000] Output: 4000</pre>	1046. Last Stone Weight (E, M, H) Given a list of stones, find the last stone weight after smashing the stones. Input: stones = [2, 7, 4] Output: 1 Time Complexity: O(N^2) Space Complexity: O(1) Example 1: <pre>Input: stones = [2, 7, 4] Output: 1</pre> Example 2: <pre>Input: stones = [2, 7, 4] Output: 1</pre>	1047. Remove All Adjacent Duplicates in String (E, M, H) Given a string, remove all adjacent duplicates. Input: s = "abcbacdcb" Output: "a" Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: s = "abcbacdcb" Output: "a"</pre> Example 2: <pre>Input: s = "abcbacdcb" Output: "a"</pre>	1048. Longest String Without Three Consecutive Repeating Characters (E, M, H) Given a string, find the longest substring without three consecutive repeating characters. Input: s = "abcbacdcb" Output: 4 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: s = "abcbacdcb" Output: 4</pre> Example 2: <pre>Input: s = "abcbacdcb" Output: 4</pre>	1049. Longest Substring Without Repeating Characters (E, M, H) Given a string, find the longest substring without repeating characters. Input: s = "abcbacdcb" Output: 5 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: s = "abcbacdcb" Output: 5</pre> Example 2: <pre>Input: s = "abcbacdcb" Output: 5</pre>	1050. Construct Binary Tree from Preorder and Inorder Traversal (M, H) Given preorder and inorder traversal of a binary tree, construct the binary tree. Input: preorder = [1, 2, 3, 4, 5], inorder = [3, 2, 4, 1, 5] Output: [1, 2, 3, 4, 5] Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: preorder = [1, 2, 3, 4, 5], inorder = [3, 2, 4, 1, 5] Output: [1, 2, 3, 4, 5]</pre> Example 2: <pre>Input: preorder = [1, 2, 3, 4, 5], inorder = [3, 2, 4, 1, 5] Output: [1, 2, 3, 4, 5]</pre>	1051. Height Checker (E, M, H) Given a list of heights, find the number of students who are not in the expected order. Input: heights = [1, 1, 4, 2, 1, 3] Output: 3 Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: heights = [1, 1, 4, 2, 1, 3] Output: 3</pre> Example 2: <pre>Input: heights = [1, 1, 4, 2, 1, 3] Output: 3</pre>	1052. Grumpy Bookstore Owner (E, M, H) Given a list of grumpy values and a list of customers, find the maximum number of customers that can be served. Input: grumpy = [1, 0, 1, 0, 1, 0], customers = [1, 0, 1, 0, 1, 0] Output: 7 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: grumpy = [1, 0, 1, 0, 1, 0], customers = [1, 0, 1, 0, 1, 0] Output: 7</pre> Example 2: <pre>Input: grumpy = [1, 0, 1, 0, 1, 0], customers = [1, 0, 1, 0, 1, 0] Output: 7</pre>	1053. Previous Permutation With One Swap (E, M, H) Given a list of numbers, find the previous permutation with one swap. Input: nums = [3, 2, 1] Output: [3, 1, 2] Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: nums = [3, 2, 1] Output: [3, 1, 2]</pre> Example 2: <pre>Input: nums = [3, 2, 1] Output: [3, 1, 2]</pre>	1054. Distant Bikes (E, M, H) Given a list of bikes, find the number of bikes that are within a certain distance. Input: bikes = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] Output: 3 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: bikes = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] Output: 3</pre> Example 2: <pre>Input: bikes = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] Output: 3</pre>	1055. Shortest Path in Binary Matrix (E, M, H) Given a binary matrix, find the shortest path from the top-left to the bottom-right. Input: grid = [[0, 1, 0, 0, 0], [0, 1, 0, 1, 0], [0, 0, 0, 0, 0], [0, 1, 1, 1, 0], [0, 0, 0, 0, 0]] Output: 6 Time Complexity: O(N^2) Space Complexity: O(N^2) Example 1: <pre>Input: grid = [[0, 1, 0, 0, 0], [0, 1, 0, 1, 0], [0, 0, 0, 0, 0], [0, 1, 1, 1, 0], [0, 0, 0, 0, 0]] Output: 6</pre> Example 2: <pre>Input: grid = [[0, 1, 0, 0, 0], [0, 1, 0, 1, 0], [0, 0, 0, 0, 0], [0, 1, 1, 1, 0], [0, 0, 0, 0, 0]] Output: 6</pre>	1056. Find Common Elements Between Two Arrays (E, M, H) Given two arrays, find the common elements. Input: arr1 = [1, 2, 3, 4, 5], arr2 = [2, 4, 6, 8] Output: [2, 4] Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: arr1 = [1, 2, 3, 4, 5], arr2 = [2, 4, 6, 8] Output: [2, 4]</pre> Example 2: <pre>Input: arr1 = [1, 2, 3, 4, 5], arr2 = [2, 4, 6, 8] Output: [2, 4]</pre>	1057. Share Candies (E, M, H) Given a list of candies, find the number of candies that can be shared. Input: candies = [1, 1, 2, 2, 3, 3, 4, 4, 5, 5] Output: 5 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: candies = [1, 1, 2, 2, 3, 3, 4, 4, 5, 5] Output: 5</pre> Example 2: <pre>Input: candies = [1, 1, 2, 2, 3, 3, 4, 4, 5, 5] Output: 5</pre>	1058. Broken Calculator (E, M, H) Given a calculator, find the minimum number of operations to reach the target value. Input: x = 5, y = 8 Output: 2 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: x = 5, y = 8 Output: 2</pre> Example 2: <pre>Input: x = 5, y = 8 Output: 2</pre>	1059. Minimum Operations to Reduce X to Zero (E, M, H) Given a list of numbers, find the minimum number of operations to reduce x to zero. Input: nums = [1, 2, 3, 4, 5], x = 10 Output: 5 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: nums = [1, 2, 3, 4, 5], x = 10 Output: 5</pre> Example 2: <pre>Input: nums = [1, 2, 3, 4, 5], x = 10 Output: 5</pre>	1060. Longest Substring Without Repeating Characters (E, M, H) Given a string, find the longest substring without repeating characters. Input: s = "abcbacdcb" Output: 5 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: s = "abcbacdcb" Output: 5</pre> Example 2: <pre>Input: s = "abcbacdcb" Output: 5</pre>	1061. Maximum Depth of Binary Tree (E, M, H) Given a binary tree, find the maximum depth. Input: root = [3, 9, 20, null, 15, 7] Output: 3 Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: root = [3, 9, 20, null, 15, 7] Output: 3</pre> Example 2: <pre>Input: root = [3, 9, 20, null, 15, 7] Output: 3</pre>	1062. Longest Substring Without Repeating Characters (E, M, H) Given a string, find the longest substring without repeating characters. Input: s = "abcbacdcb" Output: 5 Time Complexity: O(N) Space Complexity: O(1) Example 1: <pre>Input: s = "abcbacdcb" Output: 5</pre> Example 2: <pre>Input: s = "abcbacdcb" Output: 5</pre>	1063. Binary Tree Zigzag Level Traversal (E, M, H) Given a binary tree, find the zigzag level traversal. Input: root = [3, 9, 20, null, 15, 7] Output: [[3], [9, 20], [15, 7]] Time Complexity: O(N) Space Complexity: O(N) Example 1: <pre>Input: root = [3, 9, 20, null, 15, 7] Output: [[3], [9, 20], [15, 7]]</pre> Example 2: <pre>Input: root = [3, 9, 20, null, 15, 7] Output: [[3], [9, 20], [15, 7]]</pre>	1064. Set Matrix Zeros (E, M, H)
---	---	---	---	--	---	--	---	--	--	---	---	---	--	--	--	---	--	--	--

#298 Binary Tree Longest Consecutive Sequence LeetCode - Simplified - WUJIC - LeetCode Time: O(N) Linear Problem Time Complexity: O(N), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, return the length of the longest consecutive sequence path where the values increase by one A consecutive sequence path is a path where the values increase by one Note that the path can start at any node in the tree, and you cannot go from node to its parent in the path. Example 1:  Input: root = [1,null,3,null,5,null,null] Output: 3 Explanation: The longest consecutive sequence path is 3->4->5, so return 3. Constraints: The number of nodes in the tree is in the range [1, 10^4] -100 <= Node.val <= 100 # LeetCode Time & BST #314 Binary Tree Vertical Order Traversal LeetCode - Simplified - WUJIC - LeetCode Time: O(N^2) - O(N log N) - O(N) Linear Problem Time Complexity: O(N^2), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N^2), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column) If nodes are in the same row and column, the order should be from left to right. Example 1:  Input: root = [3,9,20,null,null,15,7] Output: [[4],[9],[15],[20],[7]] Explanation: The nodes in the tree are: 4, 9, 15, 20, 7. The vertical order traversal is [[4],[9],[15],[20],[7]]. Constraints: The number of nodes in the tree is in the range [1, 1000] -100 <= Node.val <= 100 # LeetCode Time & BST #333 Largest BST Subtree LeetCode - Simplified - WUJIC - LeetCode Dynamic Programming - Time: O(N^2) - O(N) Linear Problem Time Complexity: O(N^2), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N^2), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes. A Binary Search Tree (BST) is a tree in which the nodes follow the following properties: The left subtree values are less than the value of the parent node The right subtree values are greater than the value of the parent node Example 1:  Input: root = [10,5,15,null,1,8,null,7] Output: 3 Explanation: The largest BST subtree is [5,1,8] with 3 nodes. Constraints: The number of nodes in the tree is in the range [1, 10^4] -100 <= Node.val <= 100 # LeetCode Time & BST #366 Find Leaves of Binary Tree LeetCode - Simplified - WUJIC - LeetCode Time: O(N) - O(N^2) Linear Problem Time Complexity: O(N^2), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N^2), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, collect a tree's nodes as you would doing a DFS. Collect all the leaf nodes. Remove all the leaf nodes and return the root of the tree. Repeat the process until the tree is empty. Example 1:  Input: root = [1,2,3,null,4,5,null,6] Output: [[4,5,6],[2,3],[1]] Explanation: The leaves of the tree are 4, 5, 6. The root of the tree after removing the leaves is 2, 3. The root of the tree after removing the leaves is 1. Constraints: The number of nodes in the tree is in the range [1, 1000] -100 <= Node.val <= 100 # LeetCode Time & BST #437 Path Sum III LeetCode - Simplified - WUJIC - LeetCode Time: O(N^2) - O(N) Linear Problem Time Complexity: O(N^2), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N^2), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, return the number of paths where the sum of the values along the path equals targetSum. The path does not need to start or end at the root or a leaf, it must go downwards (i.e., traveling only from parent nodes to child nodes). Example 1:  Input: root = [10,5,15,null,1,8,null,7], targetSum = 8 Output: 3 Explanation: The paths that sum to 8 are: [1,8], [5,1,8], and [10,5,7]. Constraints: The number of nodes in the tree is in the range [1, 1000] -1000 <= Node.val <= 1000 # LeetCode Time & BST #454 Boundary of Binary Tree LeetCode - Simplified - WUJIC - LeetCode Time: O(N) Linear Problem Time Complexity: O(N), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, return the values of its boundary. The boundary of a binary tree is the concatenation of the root, the left boundary (if there is any), the leaves, and the right boundary (if there is any). The left boundary is the set of nodes defined by the following: The root node's left child is in the left boundary If the root node has a left child and a right child, then the left child is in the left boundary If the root node has only a right child, then the right child is in the left boundary The right boundary is the set of nodes defined by the following: The root node's right child is in the right boundary If the root node has a left child and a right child, then the right child is in the right boundary If the root node has only a left child, then the left child is in the right boundary The leaves are the set of nodes whose left and right children are null Example 1:  Input: root = [1,2,3,null,4,5,null,6] Output: [1,2,4,5,6,3] Explanation: The boundary of the tree is [1,2,4,5,6,3]. Constraints: The number of nodes in the tree is in the range [1, 10^4] -10^4 <= Node.val <= 10^4 # LeetCode Time & BST #549 Binary Tree Longest Consecutive Sequence II LeetCode - Simplified - WUJIC - LeetCode Time: O(N^2) - O(N) Linear Problem Time Complexity: O(N^2), where N is the number of nodes in the tree, time is the time to traverse the tree, space complexity is O(N^2), where N is the height of the tree in the worst case of a skewed tree, so $\frac{N}{2}$ Problem Given the root of a binary tree, return the length of the longest consecutive sequence path in the tree. A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing. For example, [2,1,3,4,5] and [5,4,3,2,1] are two valid consecutive paths, but the path [3,2,4,1] is not valid. On the other hand, the path can be in the child-parent-child order, when not necessarily be parent-child order. Example 1:  Input: root = [1,2,3,null,4,5,null,6] Output: 4 Explanation: The longest consecutive path is [2,1,3,4], so return 4. Constraints: The number of nodes in the tree is in the range [1, 10^4] -10^4 <= Node.val <= 10^4 # LeetCode
--

